

HƯỚNG DẪN THỰC HÀNH ORACLE SQL/PL/SQL - BUỔI 5

View Nâng Cao · Trigger Phức Tạp · Procedure Chuyên Sâu · Package

BỮA TIỆC CUỐI CÙNG CỦA MÔN !!!

PHẦN 1: HỆ THỐNG KIẾN THỨC MỚI TRONG BUỔI 5

1.1. View Nâng Cao - Các Kỹ Thuật Chưa Học Trong Buổi 4

Hàm Phân Tích Cửa Sổ (Window Functions / Analytic Functions)

Hàm phân tích tính toán trên một 'cửa sổ' (window) các hàng liên quan đến hàng hiện tại - khác với hàm gộp nhóm, chúng không thu gọn hàng.

```
-- Cu pháp chung
ten_ham() OVER ([PARTITION BY cot1] [ORDER BY cot2] [khung_cua_so])

-- Khung cua so (frame):
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- Tu dau den hien tai
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING           -- 1 hang truoc + 1 sau

-- Cac ham phan tich hay dung:
RANK()          OVER (ORDER BY salary DESC)      -- Xep hang, co khoang (1,1,3)
DENSE_RANK()    OVER (ORDER BY salary DESC)      -- Xep hang lien tuc (1,1,2)
ROW_NUMBER()    OVER (ORDER BY salary DESC)      -- So thu tu tuyet doi (1,2,3)
LAG(salary,1)   OVER (ORDER BY hire_date)        -- Gia tri hang truoc do
LEAD(salary,1)  OVER (ORDER BY hire_date)        -- Gia tri hang sau do
SUM(salary)     OVER (ORDER BY hire_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) -- Tong tich luy
AVG(salary)     OVER (PARTITION BY dept_id)      -- Trung binh theo nhom (giu hang)

-- Ham phan vi (Percentile):
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY finalgrade) -- Trung vi (P50)
PERCENTILE_DISC(0.25) WITHIN GROUP (ORDER BY finalgrade) -- P25 (phan vi roi rac)
STDDEV(finalgrade) -- Do lech chuan
VARIANCE(finalgrade) -- Phuong sai
```

UNION ALL — Kết Hợp Nhiều Truy Vấn

UNION ALL gộp kết quả của nhiều SELECT (giữ nguyên trùng lặp). UNION loại trùng lặp nhưng chậm hơn.

```
-- Kiểm tra toàn vẹn dữ liệu bằng UNION ALL
SELECT 'Loi 1: SV không tồn tại' AS loi, e.studentid AS ma_tham_chieu
FROM   enrollment e WHERE NOT EXISTS (SELECT 1 FROM student s WHERE
    s.studentid=e.studentid)
UNION ALL
SELECT 'Loi 2: Lớp không tồn tại', e.classid
FROM   enrollment e WHERE NOT EXISTS (SELECT 1 FROM class c WHERE
    c.classid=e.classid)
UNION ALL
SELECT 'Loi 3: Điểm không khớp', g.studentid
FROM   grade g JOIN enrollment e ON g.studentid=e.studentid AND
    g.classid=e.classid
WHERE  g.grade != e.finalgrade;
```

PIVOT — Chuyển Hàng Thành Cột

```
-- Cách 1: Conditional Aggregation (hoạt động trên mọi phiên bản Oracle)
SELECT studentid,
       MAX(CASE WHEN classid=1 THEN finalgrade END) AS lop_1,
       MAX(CASE WHEN classid=2 THEN finalgrade END) AS lop_2,
       MAX(CASE WHEN classid=3 THEN finalgrade END) AS lop_3
FROM   enrollment
GROUP BY studentid;

-- Cách 2: Mệnh đề PIVOT (Oracle 11g+)
SELECT * FROM enrollment
PIVOT (MAX(finalgrade) FOR classid IN (1 AS lop_1, 2 AS lop_2, 3 AS lop_3));
```

1.2. Stored Procedure - Kỹ Thuật Chuyên Sâu

SYS_REFCURSOR - Con Trỏ Tham Chiếu

SYS_REFCURSOR cho phép trả về tập kết quả (bảng dữ liệu) từ một thủ tục:

```
-- Thủ tục trả về cursor
CREATE OR REPLACE PROCEDURE get_data
    (p_id      IN   NUMBER,
     p_result OUT SYS_REFCURSOR)
IS
BEGIN
    OPEN p_result FOR
        SELECT * FROM bang WHERE id = p_id;
    -- Không CLOSE ở đây! Người gọi sẽ CLOSE sau khi dùng xong
END;
/

-- Gọi và đọc kết quả:
DECLARE
    v_cur SYS_REFCURSOR;
```

```

        v_row      bang%ROWTYPE;
BEGIN
    get_data(1, v_cur);
    LOOP
        FETCH v_cur INTO v_row;
        EXIT WHEN v_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_row.ten);
    END LOOP;
    CLOSE v_cur;
END;
/

```

User-Defined Exception - Ngoại Lệ Tùy Chỉnh

```

DECLARE
    -- Khai bao exception
    ex_khong_tim_thay  EXCEPTION;
    ex_da_ton_tai      EXCEPTION;

    -- Lien ket voi ma loi (tuy chon nhung khuyen dung)
    PRAGMA EXCEPTION_INIT(ex_khong_tim_thay, -20101);
    PRAGMA EXCEPTION_INIT(ex_da_ton_tai,      -20102);

BEGIN
    -- Kich hoat exception
    IF v_count = 0 THEN
        RAISE ex_khong_tim_thay;      -- Cach 1: RAISE ten_exception
    END IF;

    -- Hoac ket hop RAISE_APPLICATION_ERROR:
    RAISE_APPLICATION_ERROR(-20101, 'Khong tim thay!'); -- Cach 2

EXCEPTION
    WHEN ex_khong_tim_thay THEN
        DBMS_OUTPUT.PUT_LINE('Bat duoc loi khong tim thay!');
    WHEN ex_da_ton_tai THEN
        DBMS_OUTPUT.PUT_LINE('Du lieu da ton tai!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Loi khac: ' || SQLERRM);
END;
/

```

Package - Đóng Gói Nhóm Đối Tượng

Package gồm 2 phần: SPEC (giao diện công khai) và BODY (hiện thực cụ thể):

```

-- Phan 1: Package SPEC (giao dien cong khai)
CREATE OR REPLACE PACKAGE pkg_ten AS
    -- Hang so cong khai
    c_max NUMBER CONSTANT := 100;

    -- Khai bao thu tuc / ham (chi chu ky, chua co code)
    PROCEDURE thu_tuc_a(p_id IN NUMBER);
    FUNCTION ham_b(p_id IN NUMBER) RETURN NUMBER;
END pkg_ten;
/

```

```
-- Phan 2: Package BODY (code thực thi)
CREATE OR REPLACE PACKAGE BODY pkg_ten AS

    -- Bien noi bo (chi dung duoc ben trong package)
    g_dem NUMBER := 0;

    PROCEDURE thu_tuc_a(p_id IN NUMBER) IS
    BEGIN
        g_dem := g_dem + 1;    -- Truy cap bien noi bo
        DBMS_OUTPUT.PUT_LINE('Da goi ' || g_dem || ' lan');
    END thu_tuc_a;

    FUNCTION ham_b(p_id IN NUMBER) RETURN NUMBER IS
        v_ket_qua NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_ket_qua FROM bang WHERE id = p_id;
        RETURN v_ket_qua;
    END ham_b;

END pkg_ten;
/

-- Goi:
BEGIN pkg_ten.thu_tuc_a(1); END; /
SELECT pkg_ten.ham_b(1) FROM DUAL;
```

BULK COLLECT và FORALL - Xử Lý Hàng Loạt Hiệu Quả

```
DECLARE
    -- Khai bao kieu mang
    TYPE t_num  IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
    TYPE t_str  IS TABLE OF VARCHAR2(50) INDEX BY PLS_INTEGER;

    v_ids  t_num;
    v_names t_str;
BEGIN
    -- BULK COLLECT: doc nhieu hang cuong la vao mang
    SELECT studentid, firstname
    BULK COLLECT INTO v_ids, v_names
    FROM student;

    -- FORALL: thuc thi DML cho tung phan tu mang (nhanh hon vong lap)
    FORALL i IN v_ids.FIRST..v_ids.LAST
        UPDATE bang SET cot = v_names(i) WHERE id = v_ids(i);

    -- FORALL voi SAVE EXCEPTIONS (tiếp tục dù có hàng bị lỗi)
    FORALL i IN v_ids.FIRST..v_ids.LAST SAVE EXCEPTIONS
        INSERT INTO log_bang VALUES (v_ids(i), v_names(i));

    DBMS_OUTPUT.PUT_LINE('Da xu ly: ' || SQL%ROWCOUNT || ' hang');
EXCEPTION
    WHEN OTHERS THEN
        -- Bat loi tu SAVE EXCEPTIONS
        FOR j IN 1..SQL%BULK_EXCEPTIONS.COUNT LOOP
```

```

        DBMS_OUTPUT.PUT_LINE('Loi hang ' ||
SQL%BULK_EXCEPTIONS(j).ERROR_INDEX
        || ': ' || SQLERRM(-
SQL%BULK_EXCEPTIONS(j).ERROR_CODE));
    END LOOP;
END;
/

```

PRAGMA AUTONOMOUS_TRANSACTION

Cho phép một thủ tục COMMIT hoặc ROLLBACK độc lập, không ảnh hưởng đến transaction của người gọi.

Thường dùng cho logging:

```

CREATE OR REPLACE PROCEDURE log_event(p_msg VARCHAR2) IS
    PRAGMA AUTONOMOUS_TRANSACTION; -- Khai bao trong IS (truoc BEGIN)
BEGIN
    INSERT INTO event_log (msg, ts) VALUES (p_msg, SYSDATE);
    COMMIT; -- COMMIT nay chi anh huong transaction nay, khong anh huong cha
END;
/

-- Ung dung: goi log_event trong trigger hoac procedure
-- Ngay ca khi transaction cha ROLLBACK, ban ghi log van ton tai

```

1.3. Trigger Nâng Cao

Compound Trigger (Oracle 11g+) - Giải Quyết Mutating Table

Compound Trigger kết hợp nhiều thời điểm kích hoạt trong một trigger duy nhất, cho phép chia sẻ biến giữa các giai đoạn:

```

CREATE OR REPLACE TRIGGER trg_compound_name
    FOR INSERT OR UPDATE OR DELETE ON ten_bang
    COMPOUND TRIGGER

    -- Bien dung chung (ton tai trong suot len thuc thi)
    TYPE t_ids IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
    v_affected_ids t_ids;
    v_idx          PLS_INTEGER := 0;

    BEFORE STATEMENT IS -- Chay 1 lan truoc toan bo thao tac
    BEGIN
        v_idx := 0;
        v_affected_ids.DELETE; -- Xoa mang
    END BEFORE STATEMENT;

    AFTER EACH ROW IS -- Chay cho moi hang bi anh huong
    BEGIN
        v_idx := v_idx + 1;
        IF INSERTING OR UPDATING THEN
            v_affected_ids(v_idx) := :NEW.classid;
        ELSE
            v_affected_ids(v_idx) := :OLD.classid;
        END IF;
    END AFTER EACH ROW;

```

```

        END IF;
    END AFTER EACH ROW;

    AFTER STATEMENT IS    -- Chạy 1 lần sau toàn bộ thao tác
    BEGIN
        -- Bây giờ có thể đọc bảng đang trigger an toàn
        FOR i IN 1..v_idx LOOP
            UPDATE class SET so_sv =
                (SELECT COUNT(*) FROM enrollment WHERE classid=v_affected_ids(i))
            WHERE classid = v_affected_ids(i);
        END LOOP;
    END AFTER STATEMENT;

END trg_compound_name;
/

```

INSTEAD OF Trigger - Cho Phép DML Trên View Phức Tạp

```

CREATE OR REPLACE TRIGGER trg_instead_of
    INSTEAD OF INSERT OR UPDATE OR DELETE
    ON ten_view                -- Phải là VIEW, không phải TABLE
    FOR EACH ROW
BEGIN
    IF INSERTING THEN
        -- Thay INSERT vào view bằng INSERT vào bảng cơ sở thực sự
        INSERT INTO bang_thuc (col1, col2) VALUES (:NEW.col1, :NEW.col2);
    ELSIF UPDATING THEN
        UPDATE bang_thuc SET col2=:NEW.col2 WHERE col1=:OLD.col1;
    ELSIF DELETING THEN
        DELETE FROM bang_thuc WHERE col1=:OLD.col1;
    END IF;
END;
/

```

DDL Trigger - Theo Dõi Thay Đổi Cấu Trúc

```

CREATE OR REPLACE TRIGGER trg_ddl
    AFTER DDL ON SCHEMA        -- Hoặc: ON DATABASE (toàn hệ thống)
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO ddl_log (event, obj_type, obj_name, ts, usr)
    VALUES (ORA_SYSEVENT,          -- 'CREATE', 'DROP', 'ALTER'...
            ORA_DICT_OBJ_TYPE,      -- 'TABLE', 'VIEW', 'PROCEDURE'...
            ORA_DICT_OBJ_NAME,      -- Tên đối tượng
            SYSDATE, USER);

    COMMIT;
END;
/

-- Các hàm DDL Trigger hay dùng:
-- ORA_SYSEVENT           : loại sự kiện (CREATE, DROP, ALTER, GRANT...)
-- ORA_DICT_OBJ_TYPE      : loại đối tượng (TABLE, VIEW, INDEX...)
-- ORA_DICT_OBJ_NAME      : tên đối tượng
-- ORA_DICT_OBJ_OWNER     : chủ sở hữu đối tượng
-- ORA_LOGIN_USER         : user thực hiện thao tác

```

Mệnh Đề WHEN Trong Trigger

```
-- WHEN loc trigger chỉ chạy khi điều kiện thỏa mãn
CREATE OR REPLACE TRIGGER trg_with_when
  AFTER UPDATE OF finalgrade ON enrollment
  FOR EACH ROW
  WHEN (NEW.finalgrade >= 50)    -- Chỉ chạy khi điểm >= 50
                                -- Lưu ý: WHEN dùng NEW không cần dấu :
BEGIN
  -- :NEW.finalgrade vẫn dùng dấu : trong BEGIN
  DBMS_OUTPUT.PUT_LINE('Điểm đạt: ' || :NEW.finalgrade);
END;
/
```

1.4. Bảng So Sánh Kỹ Thuật Mới Buổi 5

Kỹ thuật	Dùng khi nào	Ưu điểm so với cách cũ
<code>SYS_REFCURSOR</code>	Procedure cần trả về tập kết quả nhiều hàng	Linh hoạt hơn tham số OUT thông thường; caller tự quyết định xử lý kết quả
<code>BULK COLLECT + FORALL</code>	Xử lý nhiều hàng trong mảng	Nhanh hơn 10-100x so với vòng lặp thông thường vì giảm context switch giữa SQL và PL/SQL
<code>AUTONOMOUS_TRANSACTION</code>	Logging, audit không phụ thuộc transaction cha	Log luôn được ghi kể cả khi transaction chính ROLLBACK
Compound Trigger	Cần chia sẻ trạng thái giữa các giai đoạn, giải quyết mutating table	Không cần package-level variable; code gọn hơn 2 trigger riêng
<code>INSTEAD OF Trigger</code>	Cho phép DML trên view phức tạp	Người dùng thấy view như bảng thực; logic xử lý ẩn bên trong trigger
<code>DDL Trigger</code>	Audit thay đổi cấu trúc database	Không cần code thủ công; tự động ghi mọi DDL
<code>User-Defined Exception</code>	Khi cần phân biệt nhiều loại lỗi nghiệp vụ	Code exception handling rõ ràng hơn WHEN OTHERS; dễ đọc và bảo trì

PHẦN 2: HƯỚNG DẪN BÀI 1 - VIEW NÂNG CAO

Câu 1.1: View kiểm tra sinh viên học vượt (chưa hoàn thành môn tiên quyết)

Phân tích: Tự tham chiếu bảng COURSE (co là môn đang học, tq là môn tiên quyết). Dùng NOT EXISTS để phát hiện sinh viên chưa có FinalGrade cho môn tiên quyết.

```
CREATE OR REPLACE VIEW vw_prerequisite_check AS
SELECT s.studentid,
       s.firstname || ' ' || s.lastname      AS ho_ten,
       co.description                        AS ten_mon,
       co.courseno,
       tq.description                        AS ten_mon_tq,
       tq.courseno                          AS courseno_tq
FROM   enrollment e
       JOIN student  s   ON e.studentid = s.studentid
       JOIN class    cl  ON e.classid   = cl.classid
       JOIN course    co  ON cl.courseno = co.courseno
       JOIN course    tq  ON co.prerequisite = tq.courseno
WHERE  co.prerequisite IS NOT NULL
      AND NOT EXISTS (
        SELECT 1
        FROM    enrollment e2
               JOIN class cl2 ON e2.classid = cl2.classid
        WHERE   e2.studentid = s.studentid
               AND cl2.courseno = co.prerequisite
               AND e2.finalgrade IS NOT NULL
      );

-- Dem truong hop hoc vuot theo tung mon:
SELECT ten_mon, courseno, COUNT(*) AS so_sv_hoc_vuot
FROM   vw_prerequisite_check
GROUP BY ten_mon, courseno
ORDER BY so_sv_hoc_vuot DESC;
```

🔗 **Giải thích:** JOIN COURSE 2 lần với alias khác nhau (co và tq) là kỹ thuật self-join bảng. NOT EXISTS hiệu quả hơn NOT IN khi dữ liệu có NULL.

Câu 1.2: View phân tích hiệu suất giảng dạy với RANK/DENSE_RANK và tỉ lệ đạt

Phân tích: Dùng SUM(CASE) để đếm có điều kiện. NULLIF tránh chia cho 0 trong tính tỉ lệ. Hàm phân tích RANK/DENSE_RANK trong Inline View.

```
CREATE OR REPLACE VIEW vw_instructor_performance AS
SELECT instructorid, ho_ten, so_lop, tong_sv,
       sv_co_diem, diem_tb, diem_max, diem_min,
       ROUND(sv_dat * 100 / NULLIF(sv_co_diem,0), 1) AS ty_le_dat_pct,
       DENSE_RANK() OVER (ORDER BY diem_tb DESC NULLS LAST) AS
hang_diem_tb,
       RANK() OVER (ORDER BY tong_sv DESC) AS hang_so_sv
FROM (
    SELECT i.instructorid,
           i.firstname || ' ' || i.lastname AS ho_ten,
           COUNT(DISTINCT cl.classid) AS so_lop,
           COUNT(e.studentid) AS tong_sv,
           COUNT(e.finalgrade) AS sv_co_diem,
           ROUND(AVG(e.finalgrade),2) AS diem_tb,
           MAX(e.finalgrade) AS diem_max,
           MIN(e.finalgrade) AS diem_min,
           SUM(CASE WHEN e.finalgrade >= 50
                    THEN 1 ELSE 0 END) AS sv_dat
    FROM   instructor i
           LEFT JOIN class cl ON i.instructorid = cl.instructorid
           LEFT JOIN enrollment e ON cl.classid = e.classid
    GROUP BY i.instructorid, i.firstname, i.lastname
);

-- Top 3 giao vien ty le SV dat cao nhat:
SELECT ho_ten, ty_le_dat_pct, sv_co_diem, diem_tb
FROM   vw_instructor_performance
WHERE  hang_diem_tb <= 3
ORDER BY hang_diem_tb;
```

- ❖ *Lưu ý: NULLIF(sv_co_diem, 0) trả về NULL thay vì 0 khi chưa có SV nào có điểm - tránh lỗi ORA-01476: divisor is equal to zero.*

Câu 1.3: View thống kê đăng ký theo tháng với tổng tích lũy SUM() OVER

Phân tích: Dùng TO_CHAR để trích năm/tháng. SUM() OVER với ROWS UNBOUNDED PRECEDING để tính tích lũy.

```
CREATE OR REPLACE VIEW vw_monthly_enrollment_stats AS
```

```

SELECT nam, thang, so_dang_ky, so_sv_moi, so_mon, diem_tb_thang,
       SUM(so_dang_ky) OVER (
           ORDER BY nam, thang
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS luy_ke_dang_ky
FROM (
    SELECT TO_CHAR(e.enrolldate, 'YYYY')          AS nam,
           TO_CHAR(e.enrolldate, 'MM')           AS thang,
           COUNT(*)                             AS so_dang_ky,
           COUNT(DISTINCT e.studentid)          AS so_sv_moi,
           COUNT(DISTINCT cl.courseno)          AS so_mon,
           ROUND(AVG(e.finalgrade), 2)          AS diem_tb_thang
    FROM   enrollment e
           JOIN class cl ON e.classid = cl.classid
    GROUP BY TO_CHAR(e.enrolldate, 'YYYY'),
             TO_CHAR(e.enrolldate, 'MM')
)
ORDER BY nam DESC, thang ASC;

SELECT * FROM vw_monthly_enrollment_stats;

```

 **Giải thích:** Hàm phân tích SUM() OVER với ROWS UNBOUNDED PRECEDING tính tổng từ hàng đầu tiên đến hàng hiện tại - đây là cách tính tổng tích lũy (running total) chuẩn trong Oracle.

Câu 1.4: View INSTEAD OF - Cho phép INSERT vào view 4 bảng

Phân tích: Bước 1: tạo view phức tạp. Bước 2: tạo INSTEAD OF INSERT trigger xử lý logic thực tế.

Bước 1 - Tạo view:

```

CREATE OR REPLACE VIEW vw_enrollment_full AS
SELECT e.studentid, e.classid, e.enrolldate, e.finalgrade,
       s.firstname || ' ' || s.lastname AS ten_sv,
       co.description                  AS ten_mon,
       i.firstname || ' ' || i.lastname AS ten_gv
FROM   enrollment e
       JOIN student s ON e.studentid = s.studentid
       JOIN class   cl ON e.classid   = cl.classid
       JOIN course  co ON cl.courseno = co.courseno
       JOIN instructor i ON cl.instructorid = i.instructorid;

```

Bước 2 - INSTEAD OF INSERT trigger:

```
CREATE OR REPLACE TRIGGER trg_iot_enrollment_full
  INSTEAD OF INSERT ON vw_enrollment_full
  FOR EACH ROW
DECLARE
  v_sv_check    NUMBER;
  v_cl_check    NUMBER;
  v_capacity    NUMBER;
  v_enrolled    NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_sv_check FROM student WHERE
studentid=:NEW.studentid;
  IF v_sv_check=0 THEN
    RAISE_APPLICATION_ERROR(-20050,'Sinh vien khong ton tai!'); RETURN;
  END IF;

  SELECT COUNT(*) INTO v_cl_check FROM class WHERE classid=:NEW.classid;
  IF v_cl_check=0 THEN
    RAISE_APPLICATION_ERROR(-20051,'Lop hoc khong ton tai!'); RETURN;
  END IF;

  SELECT capacity INTO v_capacity FROM class WHERE classid=:NEW.classid;
  SELECT COUNT(*) INTO v_enrolled FROM enrollment WHERE
classid=:NEW.classid;
  IF v_enrolled >= v_capacity THEN
    RAISE_APPLICATION_ERROR(-20052,'Lop da day!'); RETURN;
  END IF;

  INSERT INTO enrollment

(studentid,classid,enrolldate,createdby,createddate,modifiedby,modifieddate
)
VALUES
  (:NEW.studentid,:NEW.classid,NVL(:NEW.enrolldate,SYSDATE),
  USER,SYSDATE,USER,SYSDATE);

  DBMS_OUTPUT.PUT_LINE('[OK] Da dang ky: SV '||:NEW.studentid||' -> Lop
'||:NEW.classid);
END trg_iot_enrollment_full;
/

-- Kiem tra INSERT qua view:
INSERT INTO vw_enrollment_full (studentid,classid)
```

```
VALUES (101, 3);

COMMIT;
```

Câu 1.5: View phân tích phân vị với PERCENTILE_CONT và STDDEV

Phân tích: Dùng hàm thống kê nâng cao: PERCENTILE_CONT, STDDEV. SUM(CASE) đếm theo nhóm điểm.

```
CREATE OR REPLACE VIEW vw_grade_distribution AS
SELECT classid, courseno, description,
       sv_A, sv_B, sv_C, sv_D, sv_F, sv_chua_co,
       p25, p50_median, p75,
       ROUND(std_dev,2) AS
do_lech_chuan,
       ROUND(std_dev / NULLIF(diem_tb,0) * 100, 2) AS he_so_bt
FROM (
  SELECT cl.classid,
         cl.courseno,
         co.description,
         SUM(CASE WHEN e.finalgrade>=90 THEN 1 ELSE 0 END) AS sv_A,
         SUM(CASE WHEN e.finalgrade>=80
                   AND e.finalgrade<90 THEN 1 ELSE 0 END) AS sv_B,
         SUM(CASE WHEN e.finalgrade>=70
                   AND e.finalgrade<80 THEN 1 ELSE 0 END) AS sv_C,
         SUM(CASE WHEN e.finalgrade>=50
                   AND e.finalgrade<70 THEN 1 ELSE 0 END) AS sv_D,
         SUM(CASE WHEN e.finalgrade<50
                   AND e.finalgrade IS NOT NULL
                               THEN 1 ELSE 0 END) AS sv_F,
         SUM(CASE WHEN e.finalgrade IS NULL THEN 1 ELSE 0 END) AS
sv_chua_co,
         PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY e.finalgrade) AS
p25,
         PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY e.finalgrade) AS
p50_median,
         PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY e.finalgrade) AS
p75,
         STDDEV(e.finalgrade) AS std_dev,
         AVG(e.finalgrade) AS diem_tb
  FROM class cl
  JOIN course co ON cl.courseno = co.courseno
  LEFT JOIN enrollment e ON cl.classid = e.classid
```

```

GROUP BY cl.classid, cl.courseno, co.description
HAVING COUNT(e.finalgrade) >= 2
);

SELECT * FROM vw_grade_distribution ORDER BY classid;

```

Câu 1.6: View PIVOT - Điểm sinh viên chuyển thành cột theo lớp

Phân tích: Dùng MAX(CASE WHEN classid = X THEN finalgrade END) để pivot thủ công. Thêm câu PIVOT thực sự để so sánh.

```

CREATE OR REPLACE VIEW vw_student_grade_pivot AS
SELECT studentid,
       ho_ten,
       MAX(CASE WHEN classid=1 THEN finalgrade END) AS diem_lop_1,
       MAX(CASE WHEN classid=2 THEN finalgrade END) AS diem_lop_2,
       MAX(CASE WHEN classid=3 THEN finalgrade END) AS diem_lop_3,
       MAX(CASE WHEN classid=4 THEN finalgrade END) AS diem_lop_4,
       ROUND(AVG(finalgrade),2) AS diem_tb_chung,
       COUNT(classid) AS tong_lop
FROM (
    SELECT e.studentid,
           s.firstname||' '||s.lastname AS ho_ten,
           e.classid,
           e.finalgrade
    FROM   enrollment e JOIN student s ON e.studentid=s.studentid
)
GROUP BY studentid, ho_ten
ORDER BY studentid;

SELECT * FROM vw_student_grade_pivot;

-- Cach 2: Dung menh de PIVOT (Oracle 11g+)
SELECT * FROM (
    SELECT e.studentid, e.classid, e.finalgrade
    FROM   enrollment e
)
PIVOT (MAX(finalgrade) FOR classid IN
       (1 AS diem_lop_1, 2 AS diem_lop_2,
        3 AS diem_lop_3, 4 AS diem_lop_4)

```

```
);
```

Câu 1.7: View kiểm tra toàn vẹn dữ liệu bằng UNION ALL

Phân tích: 4 SELECT riêng, mỗi cái phát hiện một loại lỗi, gộp bằng UNION ALL. Mỗi SELECT có cùng cấu trúc cột.

```
CREATE OR REPLACE VIEW vw_data_integrity_check AS
-- Lỗi 1: SV trong ENROLLMENT không tồn tại trong STUDENT
SELECT 'LOI_1: SV_KHONG_TON_TAI'      AS loại_vấn_đề,
       TO_CHAR(e.studentid)            AS mã_tham_chiếu,
       'StudentID '||e.studentid||' có trong ENROLLMENT nhưng không có
       trong STUDENT'
       AS mô_tả
FROM   enrollment e
WHERE  NOT EXISTS (SELECT 1 FROM student s WHERE s.studentid=e.studentid)

UNION ALL

-- Lỗi 2: Lớp trong CLASS thiếu giáo viên
SELECT 'LOI_2: LOP_THIEU_GIAO_VIEN',
       TO_CHAR(cl.classid),
       'ClassID '||cl.classid||' không có InstructorID hợp lệ'
FROM   class cl
WHERE  NOT EXISTS (SELECT 1 FROM instructor i WHERE
i.instructorid=cl.instructorid)

UNION ALL

-- Lỗi 3: Điểm trong GRADE không khớp với ENROLLMENT
SELECT 'LOI_3: DIEM_KHONG_KHOP',
       TO_CHAR(g.studentid)|| '/' || TO_CHAR(g.classid),
       'GRADE.grade='||g.grade||' khác
       ENROLLMENT.finalgrade='||e.finalgrade
FROM   grade g
       JOIN enrollment e ON g.studentid=e.studentid AND g.classid=e.classid
WHERE  g.grade != NVL(e.finalgrade,-999)

UNION ALL

-- Lỗi 4: SV đang ký quá 3 lớp
```

```
SELECT 'LOI_4: DANG_KY_QUA_3_LOP',  
       TO_CHAR(studentid),  
       'StudentID '||studentid||' dang ky '||COUNT(*)||' lop (toi da 3)'  
FROM   enrollment  
GROUP BY studentid  
HAVING COUNT(*) > 3;  
  
SELECT * FROM vw_data_integrity_check;
```

PHẦN 3: HƯỚNG DẪN BÀI 2 - STORED PROCEDURE NÂNG CAO

Câu 2.1: Thủ tục trả về SYS_REFCURSOR + thủ tục in kết quả

Phân tích: OPEN p_result FOR SELECT... - không đóng cursor trong thủ tục. Thủ tục gọi nhận cursor rồi FETCH vòng lặp.

Thủ tục trả về cursor:

```
CREATE OR REPLACE PROCEDURE get_students_by_class  
    (p_classid IN NUMBER,  
     p_result OUT SYS_REFCURSOR)  
IS  
    v_check NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_check FROM class WHERE classid=p_classid;  
    IF v_check=0 THEN  
        p_result := NULL;  
        DBMS_OUTPUT.PUT_LINE('Lop '||p_classid||' khong ton tai!');  
        RETURN;  
    END IF;  
  
    OPEN p_result FOR  
        SELECT s.studentid,  
               s.firstname||' '||s.lastname AS ho_ten,  
               e.finalgrade,  
               CASE  
                   WHEN e.finalgrade>=90 THEN 'A'
```

```

        WHEN e.finalgrade>=80 THEN 'B'
        WHEN e.finalgrade>=70 THEN 'C'
        WHEN e.finalgrade>=50 THEN 'D'
        WHEN e.finalgrade IS NULL THEN 'Chua co'
        ELSE 'F'
    END AS xep_loai,
    RANK() OVER (ORDER BY e.finalgrade DESC NULLS LAST) AS
thu_hang
FROM   enrollment e
       JOIN student s ON e.studentid=s.studentid
       WHERE e.classid=p_classid
       ORDER BY thu_hang;
END get_students_by_class;
/

```

Thủ tục in kết quả:

```

CREATE OR REPLACE PROCEDURE print_class_result
    (p_classid IN NUMBER)
IS
    v_cur      SYS_REFCURSOR;
    v_sid      NUMBER;
    v_ten      VARCHAR2(50);
    v_diem     NUMBER;
    v_xep      VARCHAR2(10);
    v_hang     NUMBER;
BEGIN
    get_students_by_class(p_classid, v_cur);
    IF v_cur IS NULL THEN RETURN; END IF;

    DBMS_OUTPUT.PUT_LINE('=== KET QUA LOP ' || p_classid || ' ===');
    DBMS_OUTPUT.PUT_LINE(RPAD('Hang',5)||RPAD('Ho
Ten',22)||LPAD('Diem',6)||' Xep loai');
    DBMS_OUTPUT.PUT_LINE(RPAD('-',45,'-'));
    LOOP
        FETCH v_cur INTO v_sid, v_ten, v_diem, v_xep, v_hang;
        EXIT WHEN v_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
            LPAD(v_hang,4)||'   '||RPAD(v_ten,22)
            ||LPAD(NVL(TO_CHAR(v_diem),'--'),6)||'   '||v_xep);
    END LOOP;
    CLOSE v_cur;

```

```

END print_class_result;

/

BEGIN print_class_result(1); END; /

```

Câu 2.2: Exception tùy chỉnh với PRAGMA EXCEPTION_INIT

Phân tích: Khai báo exception trong DECLARE, gán PRAGMA, RAISE trong BEGIN, bắt trong EXCEPTION.

```

CREATE OR REPLACE PROCEDURE validate_enrollment
    (p_studentid IN NUMBER, p_classid IN NUMBER)
IS
    ex_sv_not_found      EXCEPTION; PRAGMA EXCEPTION_INIT(ex_sv_not_found, -
    20101);

    ex_class_not_found EXCEPTION; PRAGMA
    EXCEPTION_INIT(ex_class_not_found, -20102);

    ex_class_full        EXCEPTION; PRAGMA EXCEPTION_INIT(ex_class_full,
    -20103);

    ex_already_enrolled EXCEPTION; PRAGMA
    EXCEPTION_INIT(ex_already_enrolled, -20104);

    v_check  NUMBER; v_cap NUMBER; v_enr NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM student WHERE studentid=p_studentid;
    IF v_check=0 THEN
        RAISE_APPLICATION_ERROR(-20101,'Sinh vien '||p_studentid||' khong
        ton tai!');
    END IF;

    SELECT COUNT(*) INTO v_check FROM class WHERE classid=p_classid;
    IF v_check=0 THEN
        RAISE_APPLICATION_ERROR(-20102,'Lop hoc '||p_classid||' khong ton
        tai!');
    END IF;

    SELECT capacity INTO v_cap FROM class WHERE classid=p_classid;
    SELECT COUNT(*) INTO v_enr FROM enrollment WHERE classid=p_classid;
    IF v_enr>=v_cap THEN
        RAISE_APPLICATION_ERROR(-20103,'Lop '||p_classid||' da day
        ('||v_enr||'/'||v_cap||')!');
    END IF;

    SELECT COUNT(*) INTO v_check FROM enrollment

```

```

WHERE studentid=p_studentid AND classid=p_classid;

IF v_check>0 THEN
    RAISE_APPLICATION_ERROR(-20104,'SV '||p_studentid||' da dang ky lop nay roi!');
END IF;

DBMS_OUTPUT.PUT_LINE('[OK] Tat ca dieu kien deu thoa man!');
EXCEPTION
    WHEN ex_sv_not_found THEN DBMS_OUTPUT.PUT_LINE('[KHONG TON TAI] '||SQLERRM);
    WHEN ex_class_not_found THEN DBMS_OUTPUT.PUT_LINE('[LOP SAI] '||SQLERRM);
    WHEN ex_class_full THEN DBMS_OUTPUT.PUT_LINE('[DAY LOP] '||SQLERRM);
    WHEN ex_already_enrolled THEN DBMS_OUTPUT.PUT_LINE('[TRUNG LAP] '||SQLERRM);
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('[LOI KHAC] '||SQLERRM);
END validate_enrollment;
/

```

Câu 2.3: Thủ tục đệ quy tính học phí cộng dồn tiên quyết

Phân tích: Thủ tục gọi lại chính nó với môn tiên quyết. Tham số p_depth kiểm soát độ sâu tối đa.

```

CREATE OR REPLACE PROCEDURE calc_total_prerequisite_cost
(
    p_courseno IN NUMBER,
    p_total OUT NUMBER,
    p_depth IN NUMBER DEFAULT 0)
IS
    v_cost NUMBER;
    v_prereq NUMBER;
    v_desc VARCHAR2(50);
    v_sub_total NUMBER := 0;
    v_indent VARCHAR2(40);
BEGIN
    IF p_depth >= 10 THEN
        DBMS_OUTPUT.PUT_LINE('CANH BAO: Dat gioi han do sau (10)!');
        p_total := 0; RETURN;
    END IF;

    BEGIN
        SELECT cost, prerequisite, description

```

```

        INTO    v_cost, v_prereq, v_desc
        FROM    course WHERE courseno=p_courseno;

    EXCEPTION

        WHEN NO_DATA_FOUND THEN

            p_total:=0; RETURN;

    END;

    v_indent := LPAD(' ', p_depth*4);

    IF v_prereq IS NOT NULL THEN
        calc_total_prerequisite_cost(v_prereq, v_sub_total, p_depth+1);
    END IF;

    p_total := NVL(v_cost,0) + v_sub_total;
    DBMS_OUTPUT.PUT_LINE(v_indent||'Cap '||p_depth||': '||p_courseno
                        ||' - '||v_desc||' (phi: '||NVL(v_cost,0)||')');
END calc_total_prerequisite_cost;
/

-- Kiem tra:
DECLARE v_total NUMBER;
BEGIN
    calc_total_prerequisite_cost(30, v_total);
    DBMS_OUTPUT.PUT_LINE('Tong hoc phi can thiet: ' || v_total);
END; /

```

Câu 2.4: Package pkg_student_mgmt - SPEC và BODY đầy đủ

Phân tích: Tạo SPEC trước (khai báo giao diện), sau đó tạo BODY (hiện thực). Biến g_log_count là trạng thái nội bộ package.

Package SPEC:

```

CREATE OR REPLACE PACKAGE pkg_student_mgmt AS

    c_max_classes CONSTANT NUMBER := 3;

    PROCEDURE enroll(p_sid NUMBER, p_cid NUMBER);
    PROCEDURE withdraw(p_sid NUMBER, p_cid NUMBER);
    FUNCTION get_student_gpa(p_sid NUMBER) RETURN NUMBER;
    PROCEDURE print_transcript(p_sid NUMBER);
    FUNCTION count_enrolled(p_sid NUMBER) RETURN NUMBER;

```

```
END pkg_student_mgmt;
```

```
/
```

Package BODY:

```
CREATE OR REPLACE PACKAGE BODY pkg_student_mgmt AS
```

```
-- Bien noi bo: dem so lan goi
```

```
g_call_count NUMBER := 0;
```

```
FUNCTION count_enrolled(p_sid NUMBER) RETURN NUMBER IS
```

```
    v_cnt NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_cnt FROM enrollment WHERE studentid=p_sid;
```

```
    RETURN v_cnt;
```

```
END;
```

```
PROCEDURE enroll(p_sid NUMBER, p_cid NUMBER) IS
```

```
    v_check NUMBER; v_cap NUMBER; v_enr NUMBER;
```

```
BEGIN
```

```
    g_call_count := g_call_count + 1;
```

```
    SELECT COUNT(*) INTO v_check FROM student WHERE studentid=p_sid;
```

```
    IF v_check=0 THEN DBMS_OUTPUT.PUT_LINE('[LOI] SV khong ton tai');  
RETURN; END IF;
```

```
    SELECT COUNT(*) INTO v_check FROM class WHERE classid=p_cid;
```

```
    IF v_check=0 THEN DBMS_OUTPUT.PUT_LINE('[LOI] Lop khong ton tai');  
RETURN; END IF;
```

```
    IF count_enrolled(p_sid) >= c_max_classes THEN
```

```
        DBMS_OUTPUT.PUT_LINE('[LOI] SV da du '||c_max_classes||' lop');  
RETURN;
```

```
    END IF;
```

```
    SELECT capacity INTO v_cap FROM class WHERE classid=p_cid;
```

```
    SELECT COUNT(*) INTO v_enr FROM enrollment WHERE classid=p_cid;
```

```
    IF v_enr>=v_cap THEN DBMS_OUTPUT.PUT_LINE('[LOI] Lop day'); RETURN;  
END IF;
```

```
    INSERT INTO  
enrollment(studentid,classid,enrolldate,createdby,createddate,modifiedby,mo  
difieddate)
```

```
VALUES (p_sid,p_cid,SYSDATE,USER,SYSDATE,USER,SYSDATE);
```

```
COMMIT;
```

```
    DBMS_OUTPUT.PUT_LINE('[OK] Dang ky: SV '||p_sid||' -> Lop  
'||p_cid);
```

```
END enroll;
```

```
PROCEDURE withdraw(p_sid NUMBER, p_cid NUMBER) IS
    v_check NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM enrollment
    WHERE studentid=p_sid AND classid=p_cid;
    IF v_check=0 THEN DBMS_OUTPUT.PUT_LINE('[LOI] SV chua dang ky lop nay'); RETURN; END IF;
    DELETE FROM enrollment WHERE studentid=p_sid AND classid=p_cid;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[OK] Da huy dang ky: SV '||p_sid||' khoi Lop '||p_cid);
END withdraw;

FUNCTION get_student_gpa(p_sid NUMBER) RETURN NUMBER IS
    v_gpa NUMBER; v_check NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM student WHERE studentid=p_sid;
    IF v_check=0 THEN RETURN NULL; END IF;
    SELECT ROUND(AVG(finalgrade),2) INTO v_gpa
    FROM enrollment WHERE studentid=p_sid AND finalgrade IS NOT NULL;
    RETURN v_gpa;
END;

PROCEDURE print_transcript(p_sid NUMBER) IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== BANG DIEM SV: '||p_sid||' ===');
    FOR rec IN (
        SELECT co.description, co.courseno, e.finalgrade,
               cl.classid
        FROM enrollment e
        JOIN class cl ON e.classid=cl.classid
        JOIN course co ON cl.courseno=co.courseno
        WHERE e.studentid=p_sid ORDER BY co.courseno
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(' '||rec.courseno||'
        ||RPAD(rec.description,25)
        ||' : '||NVL(TO_CHAR(rec.finalgrade),'Chua co diem'));
    END LOOP;
```

```

        DBMS_OUTPUT.PUT_LINE(' GPA:
'||NVL(TO_CHAR(get_student_gpa(p_sid)), 'N/A'));

    END;

END pkg_student_mgmt;
/

-- Kiem tra package:
BEGIN
    pkg_student_mgmt.enroll(101, 5);
    pkg_student_mgmt.print_transcript(101);
END; /

```

Câu 2.5: BULK COLLECT và FORALL xử lý hàng loạt

Phân tích: Khai báo TYPE IS TABLE OF. BULK COLLECT đọc nhanh vào mảng. FORALL thực thi DML nhanh. SAVE EXCEPTIONS tiếp tục khi có lỗi.

```

CREATE OR REPLACE PROCEDURE bulk_update_grades IS
    TYPE t_num IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
    v_sids    t_num;
    v_cids    t_num;
    v_grades  t_num;
    v_start   NUMBER;
    v_end     NUMBER;
    v_rows    NUMBER := 0;
BEGIN
    v_start := DBMS_UTILITY.GET_TIME;

    -- Doc du lieu vao mang nhanh
    SELECT studentid, classid, finalgrade
    BULK COLLECT INTO v_sids, v_cids, v_grades
    FROM    enrollment WHERE finalgrade IS NOT NULL;

    DBMS_OUTPUT.PUT_LINE('Doc duoc '||v_sids.COUNT||' ban ghi...');

    -- Cap nhat hang loạt vao GRADE (MERGE thay the INSERT+UPDATE)
    FORALL i IN 1..v_sids.COUNT SAVE EXCEPTIONS
        MERGE INTO grade g
        USING (SELECT v_sids(i) AS sid, v_cids(i) AS cid,

```

```

        v_grades(i) AS gr FROM DUAL) src
    ON (g.studentid=src.sid AND g.classid=src.cid)
    WHEN MATCHED THEN
        UPDATE SET g.grade=src.gr, g.modifiedby=USER,
        g.modifieddate=SYSDATE
    WHEN NOT MATCHED THEN
        INSERT
        (studentid,classid,grade,createdby,createddate,modifiedby,modifieddate)
        VALUES (src.sid,src.cid,src.gr,USER,SYSDATE,USER,SYSDATE);

v_rows := SQL%ROWCOUNT;
COMMIT;

v_end := DBMS_UTILITY.GET_TIME;
DBMS_OUTPUT.PUT_LINE('Xu ly: '||v_rows||' hang | Thoi gian: '
||ROUND((v_end-v_start)/100,2)||' giay');
EXCEPTION
    WHEN OTHERS THEN
        FOR j IN 1..SQL%BULK_EXCEPTIONS.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('Loi hang
            '||SQL%BULK_EXCEPTIONS(j).ERROR_INDEX
            ||': '||SQLERRM(-SQL%BULK_EXCEPTIONS(j).ERROR_CODE));
        END LOOP;
        ROLLBACK;
END bulk_update_grades;
/
BEGIN bulk_update_grades; END; /

```

Câu 2.6: Thủ tục báo cáo môn học với định dạng bảng ASCII

Phân tích: LPAD/RPAD căn chỉnh cột. TO_CHAR định dạng số. Cursor FOR LOOP duyệt lớp của môn.

```

CREATE OR REPLACE PROCEDURE generate_course_report
    (p_courseno IN NUMBER)
IS
    v_check NUMBER; v_desc VARCHAR2(50);
    v_cost NUMBER; v_prereq NUMBER;
    v_tong_sv NUMBER:=0; v_sum_d NUMBER:=0; v_co_d NUMBER:=0;
    v_sep VARCHAR2(70) := RPAD('=',60,'=');
    v_sep2 VARCHAR2(70) := RPAD('-',60,'-');

```

```

BEGIN
    SELECT COUNT(*) INTO v_check FROM course WHERE courseno=p_courseno;
    IF v_check=0 THEN
        DBMS_OUTPUT.PUT_LINE('Mon hoc ' || p_courseno || ' khong ton tai!');
    RETURN;
    END IF;

    SELECT description, cost, prerequisite INTO v_desc, v_cost, v_prereq
    FROM course WHERE courseno=p_courseno;

    DBMS_OUTPUT.PUT_LINE(v_sep);
    DBMS_OUTPUT.PUT_LINE('BAO CAO MON HOC: ' || p_courseno);
    DBMS_OUTPUT.PUT_LINE(v_sep2);
    DBMS_OUTPUT.PUT_LINE('Ten mon      : ' || v_desc);
    DBMS_OUTPUT.PUT_LINE('Hoc phi      : ' || TO_CHAR(NVL(v_cost,0), '999,990.00') || ' VND');
    DBMS_OUTPUT.PUT_LINE('Mon tien q: ' || NVL(TO_CHAR(v_prereq), 'Khong co'));
    DBMS_OUTPUT.PUT_LINE(v_sep2);
    DBMS_OUTPUT.PUT_LINE(RPAD('Lop',5) || RPAD('Giao vien',20)
        || LPAD('SVDK',6) || LPAD('DTB',7) || '   Trang thai');
    DBMS_OUTPUT.PUT_LINE(v_sep2);

    FOR rec IN (
        SELECT cl.classid, cl.capacity,
               i.firstname || ' ' || i.lastname AS ten_gv,
               COUNT(e.studentid) AS so_sv,
               ROUND(AVG(e.finalgrade),1) AS dtb
        FROM   class cl
               JOIN instructor i ON cl.instructorid=i.instructorid
               LEFT JOIN enrollment e ON cl.classid=e.classid
        WHERE  cl.courseno=p_courseno
        GROUP BY cl.classid,cl.capacity,i.firstname,i.lastname
        ORDER BY cl.classid
    ) LOOP
        v_tong_sv := v_tong_sv + rec.so_sv;
        IF rec.dtb IS NOT NULL THEN
            v_sum_d:=v_sum_d+rec.dtb; v_co_d:=v_co_d+1;
        END IF;
        DBMS_OUTPUT.PUT_LINE(
            LPAD(rec.classid,4) || '   ' || RPAD(rec.ten_gv,20)

```

```

        ||LPAD(rec.so_sv,5)||LPAD(NVL(TO_CHAR(rec.dtb),'--'),7)
        ||'  '||CASE WHEN rec.capacity-rec.so_sv>0
                        THEN 'Con '||(rec.capacity-rec.so_sv)||' cho'
                        ELSE 'Het cho' END);

    END LOOP;

    DBMS_OUTPUT.PUT_LINE(v_sep2);

    DBMS_OUTPUT.PUT_LINE('Tong SV dang ky : '||v_tong_sv);

    IF v_co_d>0 THEN
        DBMS_OUTPUT.PUT_LINE('Diem TB toan mon:
        '||ROUND(v_sum_d/v_co_d,2));
    END IF;

    DBMS_OUTPUT.PUT_LINE(v_sep);

END generate_course_report;

/

BEGIN generate_course_report(10); END; /

```

Câu 2.7: Hàm convert_to_gpa_40 và thủ tục print_gpa_report

Phân tích: CASE WHEN chi tiết để chuyển điểm theo thang 4.0. Tính trọng số với số tín chỉ cố định là 3.

```

CREATE OR REPLACE FUNCTION convert_to_gpa_40
    (p_studentid IN NUMBER) RETURN NUMBER
IS
    v_check NUMBER; v_gpa NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM student WHERE studentid=p_studentid;
    IF v_check=0 THEN RETURN NULL; END IF;

    SELECT ROUND(
        SUM(CASE
            WHEN finalgrade>=90 THEN 4.0
            WHEN finalgrade>=85 THEN 3.7
            WHEN finalgrade>=80 THEN 3.3
            WHEN finalgrade>=75 THEN 3.0
            WHEN finalgrade>=70 THEN 2.7
            WHEN finalgrade>=65 THEN 2.3
            WHEN finalgrade>=60 THEN 2.0
            WHEN finalgrade>=50 THEN 1.0
            ELSE 0.0
        END * 3) /    -- So tin chi = 3

```

```

        NULLIF(SUM(CASE WHEN finalgrade IS NOT NULL THEN 3 ELSE 0 END), 0)
    , 2) INTO v_gpa
FROM enrollment WHERE studentid=p_studentid;

RETURN v_gpa;
END convert_to_gpa_40;
/

CREATE OR REPLACE PROCEDURE print_gpa_report IS
BEGIN
    DBMS_OUTPUT.PUT_LINE(RPAD('StudentID',12)||RPAD('Ho Ten',25)||LPAD('GPA
(4.0)',10));
    DBMS_OUTPUT.PUT_LINE(RPAD('-',48,'-'));
    FOR rec IN (
        SELECT studentid, firstname||' '||lastname AS ho_ten
        FROM student ORDER BY studentid
    ) LOOP
        DECLARE v_gpa NUMBER;
        BEGIN
            v_gpa := convert_to_gpa_40(rec.studentid);
            IF v_gpa IS NOT NULL THEN
                DBMS_OUTPUT.PUT_LINE(
                    LPAD(rec.studentid,10)||' '||RPAD(rec.ho_ten,25)
                    ||LPAD(v_gpa,9));
            END IF;
        END;
    END LOOP;
END print_gpa_report;
/
BEGIN print_gpa_report; END; /

```

Câu 2.8: Thủ tục ghi log với PRAGMA AUTONOMOUS_TRANSACTION

Phân tích: PRAGMA AUTONOMOUS_TRANSACTION trong IS/AS (trước BEGIN). COMMIT bên trong không ảnh hưởng transaction cha.

```

CREATE TABLE notification_log (
    log_id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nguoi_nhan      VARCHAR2(50),
    noi_dung        VARCHAR2(500),
    loai            VARCHAR2(20),

```

```
thoi_gian    DATE DEFAULT SYSDATE,
trang_thai   VARCHAR2(10) DEFAULT 'SENT'
);
/

CREATE OR REPLACE PROCEDURE log_notification
(
    p_nguoi_nhan VARCHAR2,
    p_noi_dung   VARCHAR2,
    p_loai       VARCHAR2 DEFAULT 'INFO')
IS
    PRAGMA AUTONOMOUS_TRANSACTION; -- Doc lap khoi transaction cha
BEGIN
    INSERT INTO notification_log (nguoi_nhan, noi_dung, loai)
    VALUES (p_nguoi_nhan, SUBSTR(p_noi_dung,1,500), p_loai);
    COMMIT; -- Commit ngay, khong phu thuoc transaction cha
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK; -- Chi rollback autonomous transaction nay
END log_notification;
/

-- Kiem tra tinh doc lap:
BEGIN
    INSERT INTO student(studentid,lastname,registrationdate,
                        createdby,createddate,modifiedby,modifieddate)
    VALUES (9999,'Test',SYSDATE,USER,SYSDATE,USER,SYSDATE);

    log_notification('Admin','Da them SV 9999','ENROLL');

    ROLLBACK; -- Rollback INSERT student, nhung log van con!
END; /

SELECT * FROM notification_log; -- Ban ghi log van con du da ROLLBACK
```

Giải thích: PRAGMA AUTONOMOUS_TRANSACTION tạo ra một sub-transaction độc lập. COMMIT/ROLLBACK trong nó chỉ ảnh hưởng đến sub-transaction đó, không ảnh hưởng gì đến transaction của người gọi.

PHẦN 4: HƯỚNG DẪN BÀI 3 - TRIGGER NÂNG CAO

Câu 3.1: Compound Trigger giải quyết Mutating Table

Phân tích: FOR INSERT OR UPDATE OR DELETE COMPOUND TRIGGER. BEFORE STATEMENT khởi tạo mảng. AFTER EACH ROW thu thập ID. AFTER STATEMENT cập nhật thống kê.

```
ALTER TABLE class ADD so_sv NUMBER DEFAULT 0;

CREATE OR REPLACE TRIGGER trg_update_class_count
FOR INSERT OR UPDATE OR DELETE ON enrollment
COMPOUND TRIGGER

TYPE t_ids IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
v_ids t_ids;
v_idx PLS_INTEGER := 0;

BEFORE STATEMENT IS
BEGIN
    v_idx := 0;
    v_ids.DELETE;
END BEFORE STATEMENT;

AFTER EACH ROW IS
BEGIN
    v_idx := v_idx + 1;
    v_ids(v_idx) := CASE
        WHEN INSERTING OR UPDATING THEN :NEW.classid
        ELSE :OLD.classid
    END;
END AFTER EACH ROW;

AFTER STATEMENT IS
BEGIN
    FOR i IN 1..v_idx LOOP
        UPDATE class
        SET so_sv = (SELECT COUNT(*) FROM enrollment
```

```

WHERE classid = v_ids(i))
WHERE classid = v_ids(i);
END LOOP;
END AFTER STATEMENT;

END trg_update_class_count;
/

-- Kiểm tra:
INSERT INTO enrollment(studentid,classid,enrolldate,createdby,
                        createddate,modifiedby,modifieddate)
VALUES (101,1,SYSDATE,USER,SYSDATE,USER,SYSDATE);
COMMIT;
SELECT classid, so_sv FROM class WHERE classid=1;

```

Giải thích: Biến v_ids được khai báo ở cấp COMPOUND TRIGGER (không phải trong một giai đoạn cụ thể) nên có thể chia sẻ giữa AFTER EACH ROW và AFTER STATEMENT - đây chính là giải pháp cho mutating table.

Câu 3.2: INSTEAD OF UPDATE Trigger trên view 4 bảng

Phân tích: INSTEAD OF UPDATE ON view. Cập nhật thực sự vào ENROLLMENT và GRADE. Gọi log_notification (autonomous transaction).

```

CREATE OR REPLACE VIEW vw_class_enrollment_detail AS
SELECT e.classid, e.studentid,
       s.firstname||' '||s.lastname AS ten_sv,
       co.description                AS ten_mon,
       e.finalgrade,
       i.firstname||' '||i.lastname AS ten_gv
FROM   enrollment e
       JOIN student s   ON e.studentid=s.studentid
       JOIN class cl    ON e.classid=cl.classid
       JOIN course co   ON cl.courseno=co.courseno
       JOIN instructor i ON cl.instructorid=i.instructorid;

CREATE OR REPLACE TRIGGER trg_iot_update_grade
INSTEAD OF UPDATE ON vw_class_enrollment_detail
FOR EACH ROW
DECLARE
    v_old_grade NUMBER;
BEGIN

```

```
IF :NEW.finalgrade IS NOT NULL AND
    (:NEW.finalgrade < 0 OR :NEW.finalgrade > 100) THEN
    RAISE_APPLICATION_ERROR(-20060,'Diem khong hop le (0-100)!');
END IF;

SELECT finalgrade INTO v_old_grade FROM enrollment
WHERE studentid=:OLD.studentid AND classid=:OLD.classid;

UPDATE enrollment
SET     finalgrade=:NEW.finalgrade,
        modifiedby=USER, modifieddate=SYSDATE
WHERE   studentid=:OLD.studentid AND classid=:OLD.classid;

MERGE INTO grade g
USING (SELECT :OLD.studentid AS sid, :OLD.classid AS cid FROM DUAL) src
ON (g.studentid=src.sid AND g.classid=src.cid)
WHEN MATCHED THEN
    UPDATE SET g.grade=:NEW.finalgrade,
               g.modifiedby=USER, g.modifieddate=SYSDATE
WHEN NOT MATCHED THEN
    INSERT
(studentid,classid,grade,createdby,createddate,modifiedby,modifieddate)
VALUES (:OLD.studentid,:OLD.classid,:NEW.finalgrade,
        USER,SYSDATE,USER,SYSDATE);

log_notification('System',
    'Cap nhat diem SV '||:OLD.studentid||' lop '||:OLD.classid
    ||': '||NVL(TO_CHAR(v_old_grade),'NULL')||' ->'||:NEW.finalgrade,
    'GRADE');
END trg_iot_update_grade;
/

-- Kiem tra:
UPDATE vw_class_enrollment_detail SET finalgrade=88
WHERE studentid=101 AND classid=1;
COMMIT;
```

Câu 3.3: DDL Trigger ghi nhật ký mọi thay đổi cấu trúc

Phân tích: AFTER DDL ON SCHEMA. Dùng PRAGMA AUTONOMOUS_TRANSACTION để log luôn được lưu. Các hàm ORA_SYSEVENT, ORA_DICT_OBJ_NAME...

```
CREATE TABLE ddl_audit_log (
    log_id          NUMBER GENERATED ALWAYS AS IDENTITY,
    event_type      VARCHAR2(30),
    object_type     VARCHAR2(30),
    object_name     VARCHAR2(128),
    owner           VARCHAR2(30),
    event_time      DATE,
    current_usr     VARCHAR2(30)
);
/

CREATE OR REPLACE TRIGGER trg_ddl_audit
    AFTER DDL ON SCHEMA
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO ddl_audit_log
        (event_type, object_type, object_name, owner, event_time,
        current_usr)
    VALUES
        (ORA_SYSEVENT,
        ORA_DICT_OBJ_TYPE,
        ORA_DICT_OBJ_NAME,
        ORA_DICT_OBJ_OWNER,
        SYSDATE,
        ORA_LOGIN_USER);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK;
END trg_ddl_audit;
/

-- Kiểm tra:
CREATE TABLE test_ddl_track (id NUMBER);
DROP TABLE test_ddl_track;
SELECT * FROM ddl_audit_log ORDER BY log_id DESC;
```

 Lưu ý: DDL Trigger không thể dùng ROLLBACK để hủy lệnh DDL đã kích hoạt nó (khác với DML Trigger BEFORE có thể hủy). Để ngăn DDL, dùng BEFORE DDL và RAISE_APPLICATION_ERROR.

Câu 3.4: Trigger chuỗi - Cascade khi xóa sinh viên

Phân tích: 3 trigger hoạt động theo chuỗi. Trigger 1 ngăn xóa khi còn enrollment. Trigger 2 cascade xóa grade. Trigger 3 cập nhật so_sv.

```
-- Trigger 1: BEFORE DELETE tren STUDENT - Kiem tra
CREATE OR REPLACE TRIGGER trg_prevent_student_delete
    BEFORE DELETE ON student FOR EACH ROW
DECLARE v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM enrollment WHERE
studentid=:OLD.studentid;

    IF v_count>0 THEN
        RAISE_APPLICATION_ERROR(-20030,
            'Khong the xoa SV '||:OLD.studentid||
            ' dang co '||v_count||' lop dang ky! Huy dang ky truoc.');
```

```
    END IF;
END;
/

-- Trigger 2: AFTER DELETE tren STUDENT - Cascade xoa GRADE
CREATE OR REPLACE TRIGGER trg_cascade_delete_grade
    AFTER DELETE ON student FOR EACH ROW
BEGIN
    DELETE FROM grade WHERE studentid=:OLD.studentid;

    DBMS_OUTPUT.PUT_LINE('Da xoa '||SQL%ROWCOUNT||' ban ghi GRADE cua SV
'||:OLD.studentid);
END;
/

-- Trigger 3: AFTER DELETE tren ENROLLMENT - Cap nhat so_sv trong CLASS
-- (Compound Trigger o cau 3.1 da xu ly dieu nay!)
-- Neu chua co compound trigger, viet them:
CREATE OR REPLACE TRIGGER trg_update_count_on_delete
    AFTER DELETE ON enrollment FOR EACH ROW
BEGIN
    UPDATE class SET so_sv=so_sv-1 WHERE classid=:OLD.classid AND so_sv>0;
```

```

END;

/

-- Kiểm tra chuỗi trigger:
-- Thu xóa SV có enrollment (bị chặn):
DELETE FROM student WHERE studentid=101;

-- Xóa enrollment trước, sau đó xóa SV (thành công):
DELETE FROM enrollment WHERE studentid=101;
DELETE FROM student WHERE studentid=101;
ROLLBACK;

```

Câu 3.5: Trigger với WHEN - Tự động cấp chứng chỉ khi đạt điểm

Phân tích: Mệnh đề WHEN (NEW.finalgrade >= 50) lọc trigger, chỉ chạy khi điểm đạt. Kiểm tra trùng lặp trước khi INSERT.

```

CREATE TABLE certificate (
    cert_id      NUMBER GENERATED ALWAYS AS IDENTITY,
    studentid    NUMBER(8),
    courseno     NUMBER(8),
    cap_cc       DATE,
    loai         VARCHAR2(20)
);

/

CREATE OR REPLACE TRIGGER trg_auto_certificate
    AFTER UPDATE OF finalgrade ON enrollment
    FOR EACH ROW
    WHEN (NEW.finalgrade >= 50)    -- Chỉ chạy khi điểm >= 50
DECLARE
    v_courseno   NUMBER;
    v_check      NUMBER;
    v_loai       VARCHAR2(20);
    v_ten_sv     VARCHAR2(50);
    v_ten_mon    VARCHAR2(50);
BEGIN
    -- Lấy thông tin
    SELECT cl.courseno, co.description, s.firstname||' '||s.lastname
    INTO    v_courseno, v_ten_mon, v_ten_sv

```

```
FROM class cl JOIN course co ON cl.courseno=co.courseno
      JOIN student s ON s.studentid=:NEW.studentid
WHERE cl.classid=:NEW.classid;

-- Kiem tra da co chung chi chua
SELECT COUNT(*) INTO v_check FROM certificate
WHERE studentid=:NEW.studentid AND courseno=v_courseno;
IF v_check>0 THEN RETURN; END IF; -- Da co roi, bo qua

-- Xac dinh loai chung chi
v_loai := CASE
    WHEN :NEW.finalgrade >= 90 THEN 'HIGH_DISTINCTION'
    WHEN :NEW.finalgrade >= 75 THEN 'DISTINCTION'
    ELSE 'PASS'
END;

-- Cap chung chi
INSERT INTO certificate (studentid, courseno, cap_cc, loai)
VALUES (:NEW.studentid, v_courseno, SYSDATE, v_loai);

DBMS_OUTPUT.PUT_LINE('Chuc mung '||v_ten_sv||' da hoan thanh mon '
                      ||v_ten_mon||' voi '||v_loai||'!');
END trg_auto_certificate;
/

-- Kiem tra:
UPDATE enrollment SET finalgrade=92 WHERE studentid=101 AND classid=1;
COMMIT;
SELECT * FROM certificate;
```

Giải thích: Mệnh đề WHEN trong trigger dùng NEW (không có dấu : chỉ dùng trong phần WHEN. Bên trong BEGIN phải dùng :NEW.cot như bình thường.

PHẦN 5: HƯỚNG DẪN BÀI 4 - TỔNG HỢP

Câu 4.1: Hệ thống đăng ký học hoàn chỉnh - Package + View + Trigger

Phân tích: Tổng hợp toàn bộ. Xây dựng theo thứ tự: View dashboard trước, rồi Package, rồi Trigger kiểm tra thứ tự đăng ký.

View vw_enrollment_dashboard:

```
CREATE OR REPLACE VIEW vw_enrollment_dashboard AS
SELECT
    (SELECT COUNT(*) FROM class) AS so_lop_mo,
    (SELECT SUM(cl.capacity-NVL(ec.sv,0))
     FROM class cl
     LEFT JOIN (SELECT classid,COUNT(*) sv FROM enrollment GROUP BY
classid) ec
     ON cl.classid=ec.classid) AS
tong_cho_trong,
    ROUND(
        (SELECT COUNT(*) FROM enrollment)*100.0/
        NULLIF((SELECT SUM(capacity) FROM class),0)
        ,1) AS
ty_le_lap_day_pct,
    (SELECT classid||' ('||sv||' SV)'
     FROM (SELECT classid, COUNT(*) sv FROM enrollment GROUP BY classid
           ORDER BY sv DESC FETCH FIRST 1 ROW ONLY)) AS
lop_dong_nhat,
    (SELECT classid||' ('||sv||' SV)'
     FROM (SELECT classid, COUNT(*) sv FROM enrollment GROUP BY classid
           ORDER BY sv ASC FETCH FIRST 1 ROW ONLY)) AS lop_it_nhat
FROM DUAL;

SELECT * FROM vw_enrollment_dashboard;
```

Package pkg_enrollment_system (rút gọn phần chính):

```
CREATE OR REPLACE PACKAGE pkg_enrollment_system AS
    FUNCTION is_eligible(p_sid NUMBER, p_cid NUMBER) RETURN BOOLEAN;
    PROCEDURE do_enroll(p_sid NUMBER, p_cid NUMBER);
    PROCEDURE do_withdraw(p_sid NUMBER, p_cid NUMBER);
    FUNCTION get_waitlist_position(p_sid NUMBER, p_cid NUMBER) RETURN
NUMBER;
END pkg_enrollment_system;

/
```

```
CREATE OR REPLACE PACKAGE BODY pkg_enrollment_system AS

    FUNCTION is_eligible(p_sid NUMBER, p_cid NUMBER) RETURN BOOLEAN IS
        v_sv NUMBER; v_cl NUMBER; v_cap NUMBER; v_enr NUMBER; v_dup NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_sv FROM student WHERE studentid=p_sid;
        IF v_sv=0 THEN RETURN FALSE; END IF;
        SELECT COUNT(*) INTO v_cl FROM class WHERE classid=p_cid;
        IF v_cl=0 THEN RETURN FALSE; END IF;
        SELECT capacity INTO v_cap FROM class WHERE classid=p_cid;
        SELECT COUNT(*) INTO v_enr FROM enrollment WHERE classid=p_cid;
        IF v_enr>=v_cap THEN RETURN FALSE; END IF;
        SELECT COUNT(*) INTO v_dup FROM enrollment
        WHERE studentid=p_sid AND classid=p_cid;
        IF v_dup>0 THEN RETURN FALSE; END IF;
        SELECT COUNT(*) INTO v_enr FROM enrollment WHERE studentid=p_sid;
        IF v_enr>=3 THEN RETURN FALSE; END IF;
        RETURN TRUE;
    END is_eligible;

    PROCEDURE do_enroll(p_sid NUMBER, p_cid NUMBER) IS
    BEGIN
        IF NOT is_eligible(p_sid, p_cid) THEN
            DBMS_OUTPUT.PUT_LINE('[TU CHOI] Khong du dieu kien dang ky!');
            RETURN;
        END IF;
        INSERT INTO
        enrollment(studentid,classid,enrolldate,createdby,createddate,
                    modifiedby,modifieddate)
        VALUES(p_sid,p_cid,SYSDATE,USER,SYSDATE,USER,SYSDATE);
        COMMIT;
        log_notification(USER,'Dang ky: SV '||p_sid||' -> Lop
        '||p_cid,'ENROLL');
        DBMS_OUTPUT.PUT_LINE('[OK] Dang ky thanh cong!');
    END do_enroll;

    PROCEDURE do_withdraw(p_sid NUMBER, p_cid NUMBER) IS
        v_check NUMBER;
    BEGIN
```

```

SELECT COUNT(*) INTO v_check FROM enrollment
WHERE studentid=p_sid AND classid=p_cid;
IF v_check=0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] SV chua dang ky lop nay!'); RETURN;
END IF;
DELETE FROM enrollment WHERE studentid=p_sid AND classid=p_cid;
COMMIT;
log_notification(USER,'Huy dk: SV '||p_sid||' khoi Lop
'||p_cid,'WITHDRAW');
DBMS_OUTPUT.PUT_LINE('[OK] Huy dang ky thanh cong!');
END do_withdraw;

FUNCTION get_waitlist_position(p_sid NUMBER, p_cid NUMBER) RETURN
NUMBER IS
    v_cap NUMBER; v_enr NUMBER;
BEGIN
    SELECT capacity INTO v_cap FROM class WHERE classid=p_cid;
    SELECT COUNT(*) INTO v_enr FROM enrollment WHERE classid=p_cid;
    IF v_enr < v_cap THEN RETURN 0; END IF;
    RETURN v_enr - v_cap + 1;
END;

END pkg_enrollment_system;
/

-- Kiem tra:
BEGIN
    pkg_enrollment_system.do_enroll(101, 3);
    pkg_enrollment_system.do_enroll(999, 1); -- SV khong ton tai
END; /

```

Câu 4.2: Tối ưu hóa: EXPLAIN PLAN và BULK COLLECT Refactor

Phân tích: EXPLAIN PLAN cho thấy execution path. INDEX tăng tốc tra cứu. BULK COLLECT thay vòng lặp cursor.

Phần A - Phân tích hiệu năng với EXPLAIN PLAN:

```

-- Xem execution plan của view
EXPLAIN PLAN FOR SELECT * FROM vw_course_summary;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

```

```
-- Tim Full Table Scan (TABLE ACCESS FULL) trong output
-- Tao INDEX tren cac cot hay dung trong JOIN/WHERE:
CREATE INDEX idx_enrollment_classid ON enrollment(classid);
CREATE INDEX idx_enrollment_studentid ON enrollment(studentid);
CREATE INDEX idx_class_courseno ON class(courseno);
CREATE INDEX idx_class_instructorid ON class(instructorid);

-- Chay lai EXPLAIN PLAN sau khi tao INDEX de so sanh
EXPLAIN PLAN FOR SELECT * FROM vw_course_summary;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Ket qua mong doi: thay 'TABLE ACCESS FULL' bang 'INDEX RANGE SCAN'
```

Phần B - Refactor cursor bằng BULK COLLECT:

```
CREATE OR REPLACE PROCEDURE report_class_detail_v2
(p_classid IN NUMBER)
IS
TYPE t_rec IS RECORD(
ho_ten          VARCHAR2(50),
finalgrade      NUMBER
);
TYPE t_recs IS TABLE OF t_rec INDEX BY PLS_INTEGER;
v_data          t_recs;
v_stt           NUMBER:=0;
BEGIN
-- BULK COLLECT thay vi cursor tung hang
SELECT s.firstname||' '||s.lastname, e.finalgrade
BULK COLLECT INTO v_data
FROM enrollment e JOIN student s ON e.studentid=s.studentid
WHERE e.classid=p_classid
ORDER BY s.lastname;

DBMS_OUTPUT.PUT_LINE('So SV: '||v_data.COUNT);
FOR i IN 1..v_data.COUNT LOOP
v_stt:=v_stt+1;
DBMS_OUTPUT.PUT_LINE(LPAD(v_stt,3)||' '||RPAD(v_data(i).ho_ten,22)
||LPAD(NVL(TO_CHAR(v_data(i).finalgrade),'--'),6));
END LOOP;
END report_class_detail_v2;
/
```

Phần C - Thủ tục chạy test tự động:

```
CREATE OR REPLACE PROCEDURE run_all_tests IS
    v_pass NUMBER:=0; v_fail NUMBER:=0;
    PROCEDURE assert(p_test VARCHAR2, p_cond BOOLEAN) IS
    BEGIN
        IF p_cond THEN
            v_pass:=v_pass+1;
            DBMS_OUTPUT.PUT_LINE('[PASS] '||p_test);
        ELSE
            v_fail:=v_fail+1;
            DBMS_OUTPUT.PUT_LINE('[FAIL] '||p_test);
        END IF;
    END;
    v_cnt NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== BAT DAU TEST ===');

    -- Test 1: enroll SV ton tai vao lop ton tai
    BEGIN
        pkg_enrollment_system.do_enroll(102, 2);
        SELECT COUNT(*) INTO v_cnt FROM enrollment
        WHERE studentid=102 AND classid=2;
        assert('Enroll hop le', v_cnt > 0);
        ROLLBACK;
    EXCEPTION WHEN OTHERS THEN assert('Enroll hop le', FALSE);
    END;

    -- Test 2: enroll SV khong ton tai -> phai that bai
    BEGIN
        pkg_enrollment_system.do_enroll(99999, 1);
        SELECT COUNT(*) INTO v_cnt FROM enrollment WHERE studentid=99999;
        assert('Chon SV khong ton tai', v_cnt = 0);
    END;

    DBMS_OUTPUT.PUT_LINE('=== KET QUA: PASS='||v_pass||' FAIL='||v_fail||'
    ===');
END run_all_tests;
/

BEGIN run_all_tests; END; /
```

PHẦN 6: BẢNG TRA CỨU NHANH VÀ LỖI THƯỜNG GẶP

6.1. Tổng Hợp Kỹ Thuật Theo Từng Câu

Câu	Kỹ thuật cốt lõi	Điểm mấu chốt cần nhớ
1.1	Self JOIN, NOT EXISTS	Join COURSE 2 lần với alias khác nhau để tham chiếu môn tiên quyết
1.2	DENSE_RANK, NULLIF, SUM(CASE)	NULLIF(mẫu, 0) tránh chia cho 0 khi tính tỉ lệ
1.3	SUM() OVER, ROWS UNBOUNDED PRECEDING	Window function tính tổng tích lũy; PARTITION BY chia theo nhóm
1.4	INSTEAD OF INSERT trigger	Trigger loại này chỉ dùng cho VIEW; không có FOR TABLE
1.5	PERCENTILE_CONT, STDDEV, VARIANCE	Hàm thống kê nâng cao; WITHIN GROUP là bắt buộc cho hàm phân vị
1.6	MAX(CASE WHEN), PIVOT	Conditional aggregation là cách PIVOT thủ công hoạt động mọi phiên bản
1.7	UNION ALL	Các SELECT phải cùng số cột và kiểu tương thích; UNION ALL nhanh hơn UNION
2.1	SYS_REFCURSOR, OPEN FOR	Không đóng cursor trong thủ tục trả về; caller phải CLOSE
2.2	PRAGMA EXCEPTION_INIT	Gán PRAGMA ngay sau khai báo exception; mã lỗi -20000 đến -20999
2.3	Đệ quy PL/SQL	Đặt giới hạn độ sâu; CONNECT BY LEVEL là giải pháp thay thế không đệ quy
2.4	Package SPEC + BODY	SPEC là hợp đồng công khai; biến trong BODY tồn tại suốt phiên làm việc
2.5	BULK COLLECT, FORALL, SAVE EXCEPTIONS	BULK COLLECT + FORALL nhanh hơn vòng lặp; SAVE EXCEPTIONS tiếp tục khi có lỗi
2.8	PRAGMA AUTONOMOUS_TRANSACTION	COMMIT trong thủ tục này không ảnh hưởng transaction của người gọi
3.1	Compound Trigger	Chia sẻ biến giữa các giai đoạn; giải quyết mutating table
3.3	DDL Trigger, ORA_SYSEVENT	Bắt buộc AUTONOMOUS_TRANSACTION; không thể ROLLBACK DDL đã xảy ra
3.5	WHEN (NEW.col) trong trigger	Mệnh đề WHEN dùng NEW không có dấu :: bên trong BEGIN vẫn phải dùng :NEW

6.2. Lỗi Phổ Biến Buổi 5

Lỗi	Nguyên nhân	Cách khắc phục
ORA-04091: Mutating table	AFTER FOR EACH ROW đọc bảng đang bị trigger	Dùng Compound Trigger hoặc BEFORE thay AFTER

ORA-01476: Divisor equal to zero	Chia cho 0 trong tính toán	Dùng NULLIF(mẫu, 0) trước khi chia
ORA-06533: Subscript beyond count	Truy cập mảng PL/SQL ngoài phạm vi	Kiểm tra arr.FIRST, arr.LAST trước khi dùng; kiểm tra arr.COUNT > 0
INSTEAD OF trigger không tác dụng	Trigger áp dụng cho bảng, không phải view	INSTEAD OF chỉ dùng ON view_name, không ON table_name
DDL Trigger không ghi log	Thiếu PRAGMA AUTONOMOUS_TRANSACTION	Thêm PRAGMA trong IS block; COMMIT bên trong trigger
BULK COLLECT lỗi TYPE	TYPE IS TABLE OF khai báo sai hoặc thiếu INDEX BY	TYPE t IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
Package BODY lỗi PLS-00201	BODY tham chiếu tên khác với SPEC	Tên thủ tục/hàm trong BODY phải khớp chính xác với SPEC

Good Luck!
Good Bye!